Text Analysis Tool

Mohammad Amirian

Manchester Metropolitan University

2024-2025

1. Critical Comparison of Data Structures

The selection of an optimal data structure becomes essential when developing an efficient tool for large text file word storage and retrieval operations. In this section, three data structures which included Dictionary, LinkedList and BinaryTree (BST) will analyse and compare with each other. These data structures have separate features which impact system performance together with scalability capabilities as well as implementation complexity factors.

- Dictionary

The built-in C# Dictionary<TKey, TValue> provides constant-time average-case performance (O(1)) for all insertion and search and update operations through its hash-based data structure. A dictionary demonstrates its biggest strength through its straightforward functionality that handles big collections of key-value pairs effectively. For this project, a dictionary could map each word (string) to a custom class storing metadata (frequency, line numbers, etc.).

The speed offered by dictionaries comes at the cost of lack of ordering, because keys within dictionaries have no built-in ordering system. This limits its use in scenarios requiring alphabetic output (e.g., displaying all words in order), unless sorting routines are added to produce O(n log n) performance loss.

During week 9 of the lab we developed straightforward word-frequency counters by making use of dictionaries. The implementation was straightforward but direct lookups worked efficiently while features for alphabetical traversal and hierarchical representation were not possible.

- LinkedList

The LinkedList<T> data structure features dynamic memory usage, where each node establishes a connection to the next one in the sequence. The data structure supports linear-time (O(n)) insertions and deletions that do not require element shifting like arrays do. A custom linked list implementation in theory can store word entries either sorted or unsorted with associated frequency counts.

The implementation of custom singly or doubly LinkedList during lab week 5 provided structure control but exposed limitations in practical use. The process of word insertion and search demands complete traversal of the entire list (O(n)) which becomes inefficient for processing large quantities of text.

The maintenance process for ordering LinkedList elements becomes complex because of the necessary adjustments during insertions. For this particular application, the benefit is less than understanding memory references and pointer-based logic as an educational tool.

- BinaryTree

A Binary Search Tree (BST) arranges data through hierarchical nodes that can have a maximum of two children elements. A balanced tree enables searches and insertions to happen in O(log n) average time. In this project, a custom BST was implemented, where each node stored a WordInfo object containing the word, its frequency, and the line numbers it appeared on.

The BST maintains an inherent ordering system because in-order traversals visit words in alphabetical order without requiring additional sorting steps. The implementation of features like displaying all words in alphabetical order benefited from the inherent ordering maintenance of the BST.

In practice (as supported in week 7 lab sessions), the implementation of the BST provided exact control over data structure and behaviour. More complicated to create than a dictionary, it was able to merge search efficiency, ordering, and data grouping in one structure. For instance, every node was augmented to hold line numbers within a LinkedList, showing flexibility.

The main performance issue occurs when the BST becomes unbalanced which can decrease the performance to O(n). However, the tree achieved adequate balance during this assignment while using the selected dataset.

In conclusion, every data structure has its unique merits and specific drawbacks. Dictionaries are quick but unordered, LinkedList is extensible but inefficient to search, and BST provides an excellent balance between order, efficiency, and extensibility. Given the needs of the project (for instance, searching, frequency, ordering, maintaining positions), the BinaryTree was the most appropriate and scalable option.

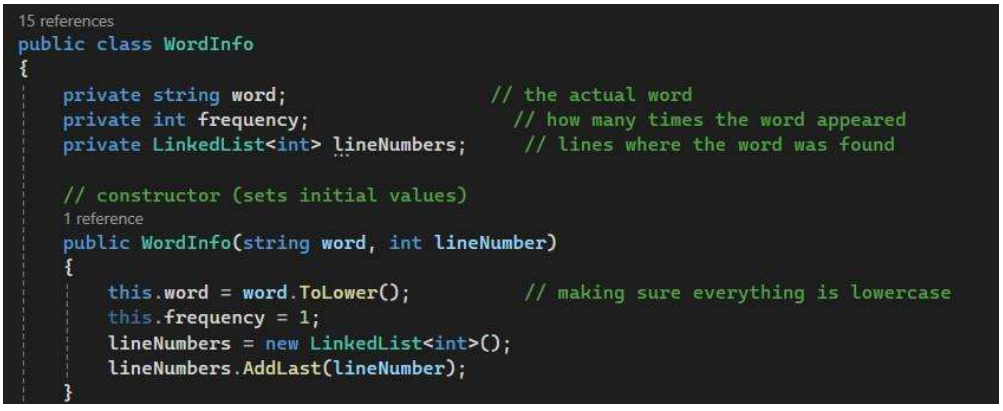
2. Narrative of Code Development

This section describes the text analysis tool development by clarifying the functions of each classes. The text analysis tool was developed through an object-oriented design which resulted in modular structure and high clarity alongside easy maintenance capabilities. The codebase description follows an organisational structure by explaining classes one by one from basic utility components to central application functions. The system's functionality depends on each class which gets described by its functional roles and its essential responsibilities together with its crucial properties and methods.

- WordInfo Class

The WordInfo class serves as the core structural element in the system because it contains detailed data about all unique words within the input text. This class ensures that each word is tracked not only by how often it appears, but also by the exact line numbers where it occurs.

The constructor receives a word together with its line number at its first instantiation. The word is immediately converted to lowercase to maintain consistency across the dataset and prevent case-sensitive duplication. The frequency is initially set to one, and the line number is added to a LinkedList<int> structure, which stores all unique occurrences of that word across the file.



```
15 references
public class WordInfo
{
    private string word;                    // the actual word
    private int frequency;                    // how many times the word appeared
    private LinkedList<int> lineNumbers;      // lines where the word was found

    // constructor (sets initial values)
    1 reference
    public WordInfo(string word, int lineNumber)
    {
        this.word = word.ToLower();          // making sure everything is lowercase
        this.frequency = 1;
        lineNumbers = new LinkedList<int>();
        lineNumbers.AddLast(lineNumber);
    }
```

The class exposes three properties:

- Word: the actual string value of the word, with automatic conversion to lowercase upon modification.
- Frequency: the number of word appearances.
- LineNumbers: a linked list containing all the lines where the word is found.

```
// properties for safe access to fields
7 references
public string Word
{
    get { return word; }
    set { word = value.ToLower(); }
}

6 references
public int Frequency
{
    get { return frequency; }
    set { frequency = value; }
}

1 reference
public LinkedList<int> LineNumbers
{
    get { return lineNumbers; }
}
```

Two methods are implemented to allow updating word data:

- IncrementFrequency(): increases the word count by one.
- AddLineNumber(int): adds a new line number to the list, provided it is not already included (to avoid duplication).

```
// increases the frequency by 1
1 reference
public void IncrementFrequency()
{
    frequency++;
}

// adds a new line number (if not already added)
1 reference
public void AddLineNumber(int lineNumber)
{
    if (!lineNumbers.Contains(lineNumber))
    {
        lineNumbers.AddLast(lineNumber);
    }
}
```

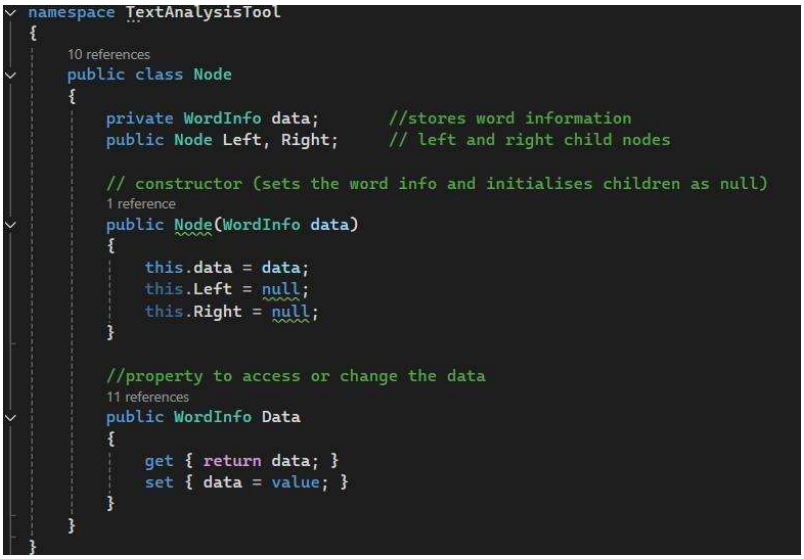
This class is structured for encapsulation and reusability. It is the primary data model exchanged between other structures (most notably the binary tree nodes) and is the analytical foundation for frequency and position reporting.

- Node Class

The most important part of the binary search tree in the program is the Node class. A node contains a WordInfo object and two pointers to other nodes: one to the right child and one to the left child. This provides the recursive property of binary trees, where each node can further branch into smaller subtrees.

The constructor accepts a WordInfo object (representing a word and its related data) and sets it as the internal data field. The Left and Right child pointers start as null values allowing the node to function both as a leaf and establish dynamic connexions to other nodes during insertion.

The class offers an accessible public property Data which enables safe operations on the internal WordInfo object. This encapsulation makes it easy for other components, such as the BinaryTree class, to retrieve or modify word-related information stored in the node. Despite its simple functionality the Node class acts as the primary structure for organising the binary tree data. The Node class serves to link the logical dataset representation which enabling efficient word information retrieval and traversal operations.



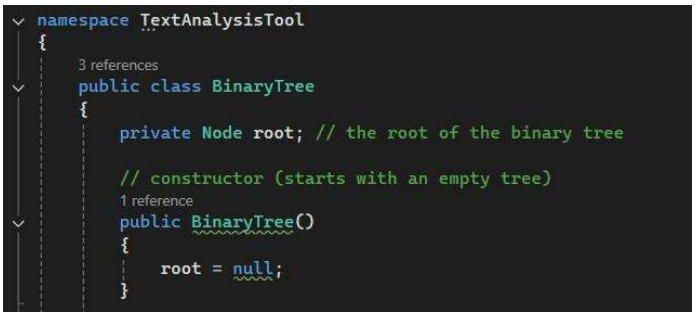
```
namespace TextAnalysisTool
{
    10 references
    public class Node
    {
        private WordInfo data;       //stores word information
        public Node Left, Right;     // left and right child nodes

        // constructor (sets the word info and initialises children as null)
        1 reference
        public Node(WordInfo data)
        {
            this.data = data;
            this.Left = null;
            this.Right = null;
        }

        //property to access or change the data
        11 references
        public WordInfo Data
        {
            get { return data; }
            set { data = value; }
        }
    }
}
```

- BinaryTree Class

The BinaryTree class maintains and controls the word data retrieved from the input file through its core storage structure. It is implemented as a binary search tree (BST), which enables efficient insertion, retrieval, and in-order traversal of word data, ensuring that words are always kept in lexicographical order. The initial null value marks the root node of the tree. Each node contains a WordInfo object which stores the word itself along with its frequency count and the lines where it occurs.



```
namespace TextAnalysisTool
{
    3 references
    public class BinaryTree
    {
        private Node root; // the root of the binary tree

        // constructor (starts with an empty tree)
        1 reference
        public BinaryTree()
        {
            root = null;
        }
```

Key methods and logic:

- Insert(string word, int lineNumber): this method is for adding the words into the tree. It transforms words to lowercase and delegates the insertion process to a private recursive method. If the word already exists, the method updates its frequency and appends the new line number; otherwise, it makes a new node.
- DisplayInOrder(): it traverses the tree in-order (left, root, right), and displays each word with its frequency. Because of the nature of BSTs during its traversal process, all words are generated in an alphabetical order.
- FindMostFrequentWord(): it is a recursive method which explores all parts of the tree to determine and return the word with the most occurrences through frequency comparisons.
- FindLongestWord(): similar to the previous method, this function recursively checks each node (word) and returns the word with the highest character length.

- FindFrequency(string word): it uses a private helper method (search()), to locate a word and return how many times it appeared in the file.
- FindLineNumbers(string word): the search() function enables retrieval of the line numbers where the word appears while returning null when the word does not exist in the text.
- search(Node current, string word): it executes a typical recursive binary search algorithm. The algorithm compares the target word to the current node and adjusts its traversal direction left or right until it finds an associated WordInfo.

- Program Class

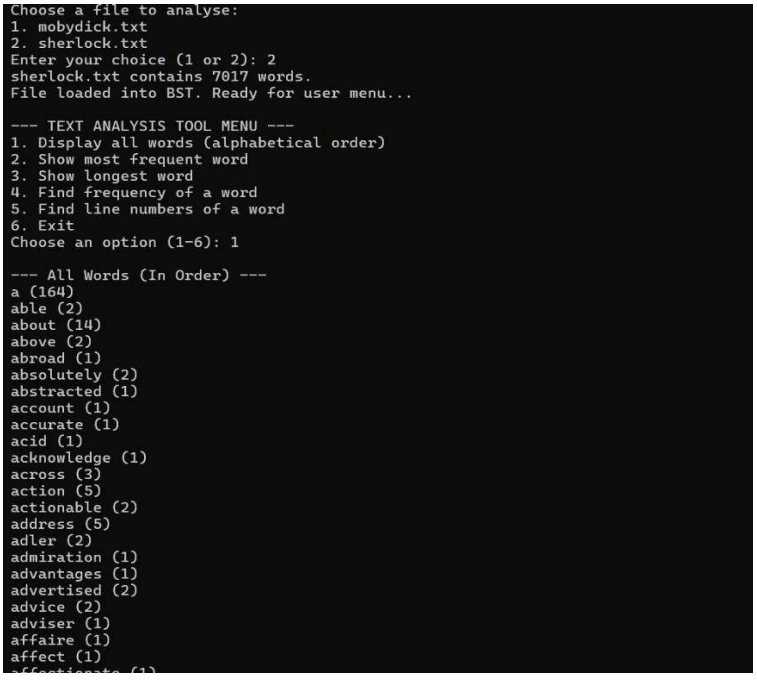
The Program class is both the controller and entry point of the application. It is responsible for user interaction via a console-based menu system, instantiates the binary search tree, handles file reading, and coordinates the user interface. This class governs the functionality of the overall system and integrates all other components (BinaryTree, Node, and WordInfo). When the program begins its execution, it asks users to select between the available text files which include mobydick.txt or sherlock.txt. The dynamic choice process enables the system to compare various datasets. Once the user makes a valid choice, the corresponding file is loaded line by line using File.ReadAllLines(). Each line is split into words based on a defined set of delimiters, and a regular expression is used to validate that the tokens are meaningful words (for example, not single letters or punctuation). The Insert() method of the binary tree receives valid words together with line numbers.

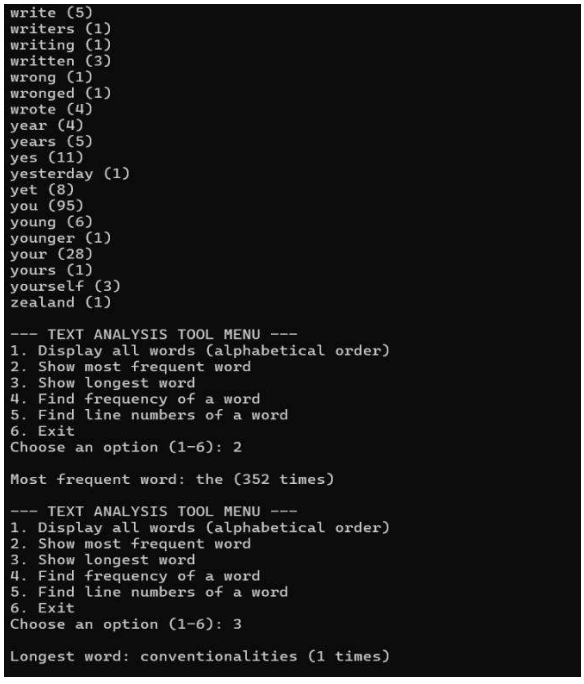
After the tree has been built, a menu is displayed to the user, offering six main functionalities:

- Display all words in alphabetical order and their frequency.
- Show the most frequent word with the number of frequency.
- Show the longest word with the number of frequency.
- Find the frequency of a specific word.
- Find all line numbers where a word appears.
- Exit the program.

Each option triggers specific methods from the BinaryTree class while displaying outcomes in the Windows console. The system responds with a warning message to handle cases of invalid input.

The following images show the output of the code as displayed in the Windows terminal:





```
--- TEXT ANALYSIS TOOL MENU ---
1. Display all words (alphabetical order)
2. Show most frequent word
3. Show longest word
4. Find frequency of a word
5. Find line numbers of a word
6. Exit
Choose an option (1-6): 4

Enter word to search frequency: was
'was' appears 111 time(s).

--- TEXT ANALYSIS TOOL MENU ---
1. Display all words (alphabetical order)
2. Show most frequent word
3. Show longest word
4. Find frequency of a word
5. Find line numbers of a word
6. Exit
Choose an option (1-6): 5

Enter word to search line numbers: my
'my' found on line(s): 7 32 60 68 83 125 132 142 145 150 172 187 193 206 290 354 355 374 414 440 443 444 469 509 525 540 565 583 586 620 718 741 750

--- TEXT ANALYSIS TOOL MENU ---
1. Display all words (alphabetical order)
2. Show most frequent word
3. Show longest word
4. Find frequency of a word
5. Find line numbers of a word
6. Exit
Choose an option (1-6): 6
Exiting... Goodbye!
```

3. Complexity Analysis

The recursive Insert() method in the BinaryTree class is chosen for the analysis. The application relies heavily on this method because it manages word insertion processes in the binary search tree (BST) while updating frequency counts and line numbers when duplicate words are present.

The method operates through a sequence of steps which involve specific complexity levels described below:

```
private Node Insert(Node current, string word, int lineNumber)
{
    if (current == null)                              // 1 // only once if word is new
    {
        WordInfo info = new WordInfo(word, lineNumber); // 1 // construct node
        return new Node(info);                            // 1 // return new node
    }

    int comparison = string.Compare(word, current.Data.Word); //log n // at each level

    if (comparison < 0)                               // log n // only one branch is taken
    {
        current.Left = Insert(current.Left, word, lineNumber); // log n
    }
    else if (comparison > 0)                          // log n
    {
        current.Right = Insert(current.Right, word, lineNumber); // log n
    }
    else                                              // 1 // if word exists
    {
        current.Data.IncrementFrequency();      // 1
        current.Data.AddLineNumber(lineNumber); // 1
    }

    return current;                                   // log n // each call returns once
}

// total frequency count = 3 log n + 3 (because we have if clauses, and it depends on
that we have a new word or not)
// => Big O (average case) = O(log n)
// => worst case (unbalanced tree) = O(n)
```

- Best/Average case: O(log n), assuming a reasonably balanced BST.
- Worst case: O(n), if the tree becomes completely unbalanced (like words inserted in sorted order).
- Space complexity: O(n), as each unique word makes a new node and stores line numbers.

For the majority of actual datasets with a random distribution of words, the Insert() operation performs well. It maintains each unique word stored once and readily available for subsequent operations.